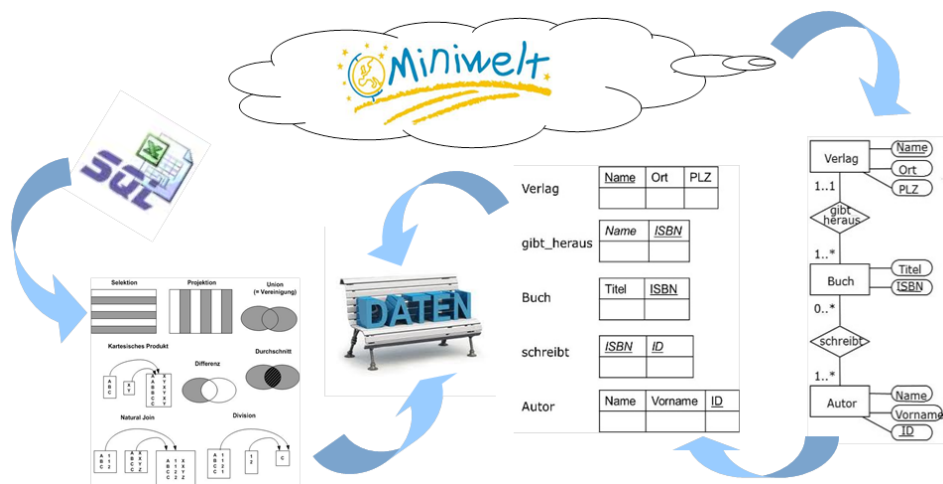


☐ DEM2 G1 Dr. Pitzer	Name _____	Aufwand in h _____
☐ DEM2 G2 Dr. Pitzer		
☐ DEM2 G3 Dr. Niklas	Punkte _____	Kurzzeichen Tutor _____

Hinweise und Richtlinien:

- Übungsausarbeitungen müssen den im Syllabus angegebenen Formatierungsrichtlinien entsprechen – Nichtbeachtung dieser Formatierungsrichtlinien führt zu Punkteabzug.
- Zusätzlich zu den allgemeinen Formatierungsrichtlinien sind für diese Übungsausarbeitung folgende zusätzlichen Richtlinien zu beachten:
 - Treffen Sie, falls notwendig, sinnvolle Annahmen und dokumentieren Sie diese nachvollziehbar in ihrer Lösung!
 - Belegen Sie die Ausführung jedes Statements und führen Sie dazu die Ausgabe der Datenbank direkt beim jeweiligen Statement in Ihrer Übungsdokumentation an.

Ziel dieser Übung ist es, die Anfragesprache SQL im Bereich Datenmanipulation (Einfügen, Löschen, Ändern von Datensätzen) praktisch zu vertiefen sowie das Sicht-Konzept anzuwenden.



1. Daten bearbeiten – INSERT, UPDATE, DELETE (Human Resources) (10 Punkte)

Die Personalabteilung möchte, dass Sie SQL-Anweisungen für das Einfügen, Aktualisieren und Löschen von Angestelltendaten erstellen. Bevor Sie die Anweisungen an die Personalabteilung übergeben, erzeugen Sie die angegebenen Tabelle MY_EMPLOYEE als Hilfstabellen, um ihre SQL-Anweisungen zu testen.

```
CREATE TABLE MY_EMPLOYEE (  
  id NUMBER(6) CONSTRAINT my_employee_id_pk PRIMARY KEY,  
  last_name VARCHAR2(25) NOT NULL,  
  first_name VARCHAR2(20) NOT NULL,  
  email VARCHAR2(8),  
  salary NUMBER(8,2) NOT NULL,  
  hire_date DATE DEFAULT SYSDATE);
```

1. Erstellen Sie zwei INSERT-Anweisungen, um der Tabelle MY_EMPLOYEE die Zeilen 1 und 2 der folgenden Beispieldaten hinzuzufügen. Listen Sie die Spalten nicht in der INSERT-Klausel auf. (1 Punkt)

ID	LAST_NAME	FIRST_NAME	EMAIL	SALARY	HIRE_DATE
1	Fawcett	Matthew	mface	2895	gestern
2	Paltrow	Vivien		3860	01.05.2024
3	Peck	Fred	fpeck	1100	01.01.2025
4	Olivier	Kevin	kolivi	2750	15.02.2025
5	Dean	John	jdean	1550	

Füllen Sie die Tabelle MY_EMPLOYEE mit den restlichen Zeilen der Beispieldaten aus der Liste in Aufgabe 1 auf. Geben Sie dieses Mal die Spalten explizit in der INSERT-Klausel an.

Kontrollieren Sie die gerade eingefügten Daten und fragen Sie dazu die Tabelle ab. Schreiben Sie dann die Daten dauerhaft fest, verwenden Sie den Befehl COMMIT. (1,5 Punkte)

2. Die Angestellte mit der ID = 5 hat geheiratet, ändern Sie den Nachnamen auf „Voight“ und passen Sie die E-Mail Adresse an („jvoigh“). (1 Punkt)
3. Alle Angestellten deren Gehalt unter 2000 liegt, erhalten Gehaltserhöhungen. Die Erhöhung beträgt 10 % vom aktuellen Gehalt; weiters wurde das Mindestgehalt auf 1250 angepasst. Führen Sie diese Änderung mit einem Datenbank-Statement aus. (1 Punkt)
4. Legen Sie für alle Angestellten, die noch keine E-Mail Adresse haben, einen solchen fest. Die Adresse setzt sich aus dem ersten Buchstaben des Vornamens und maximal fünf Buchstaben des Nachnamens zusammen (Kleinbuchstaben). (1,5 Punkte)
5. Löschen Sie „Fred Peck“ aus der Tabelle MY_EMPLOYEE und schreiben Sie alle noch nicht gespeicherten Änderungen mit COMMIT fest. (0,5 Punkte)
6. Angestellte, die bereits mindestens zwei Job-Wechsel hinter sich haben, sollen einen Bonus von 500 bekommen. Allerdings dürfen Sie aktuell keinen Job (job_id) versehen, den Sie bereits in der Vergangenheit einmal übernommen hatten. Gehen Sie dabei folgendermaßen vor:
 - a. Befüllen Sie die Tabelle MY_EMPLOYEE mit den existierenden Datensätzen aus der Tabelle EMPLOYEES. Erstellen Sie eine geeignete INSERT-Anweisung zur Übernahme der Datensätze. Überprüfen Sie danach, ob alle Datensätze enthalten sind und schreiben Sie diese dauerhaft fest. (0,5 Punkte)
 - b. Erstellen Sie nun eine SQL-Anweisung, um jene Zeilen zu aktualisieren, die mindestens zwei Einträge in der Tabelle JOB_HISTORY aufweisen. Prüfen Sie auch, ob der aktuelle Job (EMPLOYEES.JOB_ID) nicht in der Tabelle JOB_HISTORY enthalten ist. Tipp: gehen Sie in zwei Schritten vor, erstellen Sie zuerst eine SELECT-Abfrage und im zweiten Schritt daraus das UPDATE. Überprüfen Sie anschließend die durchgeführte Aktualisierung. (1,5 Punkte)
7. Die Angestellten „Hunold“ und „De Haan“ werden pensioniert. Dabei entdecken Sie, dass sich auch noch Angestellte mit einem älteren Einstellungsdatum in der Datenbank befinden, die bereits alle in Pension sind. Bereinigen Sie alle Datensätze, deren Einstellungsdatum gleich oder vor diesen beiden Angestellten ist und löschen Sie diese mit einem DELETE-Befehl. Überprüfen Sie die Löschung mit einem SELECT. (1,5 Punkte)

2. Daten bearbeiten – MERGE (Datenbankschema: Human Resources)

(4 Punkte)

Erstellen Sie für die Aufgabe 2 die Tabelle BONUS mit folgendem Schema:

```
CREATE TABLE bonus (  
  employee_id NUMBER(6) CONSTRAINT bonus_employee_id_pk PRIMARY KEY,  
  bonus NUMBER(6,2) DEFAULT 100);
```

Fügen Sie in die Tabelle BONUS mit der angeführten INSERT-Anweisung Datensätze ein und schreiben Sie die eingefügten Datensätze mit COMMIT in der Datenbank dauerhaft fest:

```
INSERT INTO bonus (employee_id)  
(SELECT employee_id  
 FROM EMPLOYEES  
 WHERE department_id IN (50, 80));
```

```
COMMIT;
```

Die Personalabteilung möchte die Bonusbeträge aller Angestellten (Tabelle EMPLOYEES) für das abgelaufene Quartal neu berechnen und die existierende Tabelle BONUS mit den bereits enthaltenen Datensätzen heranziehen und entsprechend aktualisieren.

Das Management trifft folgende Vorgaben für die Bonusberechnung:

- Bonusbeträge werden generell nur jenen Angestellten zuerkannt, die ein Gehalt unter \$11.000 beziehen – dies trifft auch auf alle Angestellten zu, die bereits im vorangegangenen Quartal Bonusbeträge bezogen haben (siehe Tabelle BONUS) – wird hier die Höchstgrenze von \$11.000 überschritten (bspw. aufgrund der Gehaltserhöhung), so sind die Datensätze der jeweiligen Angestellten aus der Tabelle BONUS zu löschen.
- Wird ein Bonusbetrag erstmalig bzw. wieder zuerkannt (d.h., es existiert kein Eintrag in der Tabelle BONUS), so beträgt dieser 1 % des aktuellen Gehaltes (EMPLOYEES.salary).
- Angestellte, die bereits im vorangegangenen Quartal einen Bonusbetrag erhalten haben (d.h., es existiert ein Eintrag in der Tabelle BONUS) und die Gehaltsgrenze von \$11.000 nicht überschreiten bekommen eine Erhöhung um 15% des aktuellen Bonusbetrages.
- Angestellte der Abteilung mit der department_id = 80 werden die Bonusbeträge aberkannt, da sie die im abgelaufenen Quartal die vorgegebenen Quartalsziele nicht erreicht haben.
- Angestellte ohne Abteilung werden wie Angestellte mit Abteilung behandelt.

Erstellen Sie eine kompakte MERGE-Anweisung, die die geforderten Einzeloperationen (UPDATE, DELETE, INSERT) in einem Durchlauf ausführt. Prüfen Sie die Anweisung und die geänderten Daten.

3. Sichten (Datenbankschema: Human Resources)

(5 Punkte)

1. Für Schulungszwecke müssen Daten von Mitarbeiter*innen zur Verfügung gestellt werden; die vertraulichen Inhalte sind dabei auszublenden. Erstellen Sie eine Sicht, die nur folgende Daten enthält: Angestellten-Nummer, Vorname, Nachname, Abteilungsname und Bundesstaat (state_province). Wählen Sie auch passende Spaltenüberschriften und selektieren Sie den Inhalt Ihrer Sicht. (1 Punkt)
2. Die Abteilung 90 benötigt Zugriff auf ihre Angestellten Daten. Erstellen Sie eine Sicht mit dem Namen DEPT90, welche die Angestelltennummern, Nachnamen und Abteilungsnummern für alle Angestellten der Abteilung 90 enthält. Die Spalten der Sicht sollen mit EMPNO, EMPLOYEE und DEPTNO benannt werden. Sorgen Sie aus Sicherheitsgründen dafür, dass kein Angestellter über die Sicht einer anderen Abteilung neu zugeordnet werden kann (= Verhinderung der Tupelmigration). Selektieren Sie den Inhalt der Sicht DEPT90. (1 Punkt)

3. Testen Sie die Sicht DEPT90. (1 Punkt)

- a) Versuchen Sie, den Angestellten „Kochhar“ der Abteilung 60 zuzuordnen.
- b) Versuchen Sie, den Angestellten 100 auf „The King“ zu ändern.
- c) Kommentieren Sie das Ergebnis und nehmen Sie die Änderungen wieder zurück.

4. Erstellen Sie eine Sicht mit den Jahresgehältern für alle Angestellte. Verketteten Sie Vor- und Nachname mit einem Leerzeichen und multiplizieren Sie das Monatsgehalt mit 14; die Sicht soll auch die Angestellten-Nummer enthalten. Testen Sie verschiedene DML-Operationen auf der Sicht, verwenden Sie dazu z.B. die Angestellten 100 und 178 und kommentieren Sie das Ergebnis. Nehmen Sie anschließend Ihre Änderungen wieder zurück. (2 Punkte)

Hinweis zur Ausarbeitung: Sie können Änderungen nicht mehr zurücknehmen, wenn Sie ein DDL-Statement absetzen (dieses führt ein implizites COMMIT durch). Bei versehentlich festgeschriebenen Änderungen, laden Sie die HR-Datenbank erneut.

4. Korrelierte Anfragen

(3 Punkte)

1. Das folgende Statement gibt alle Informationen zu Angestellten aus, die vor ihrer*m (direkten) Manager*in eingestellt wurden. Entwickeln Sie eine nicht-korrelierte Variante. (1 Punkt)

```
SELECT *
FROM employees e
WHERE EXISTS (SELECT *
              FROM employees m
              WHERE e.manager_id = m.employee_id
              AND e.hire_date < m.hire_date);
```

2. Im folgenden Statement sind zwei korrelierte Unterabfragen enthalten. Wandeln Sie beide Unterabfragen so um, dass nur mehr nicht-korrelierte Unterabfragen enthalten sind. Beachten Sie, dass ein korreliertes Statement im SELECT ev. NULL-Werte zurückgeben könnte! (2 Punkte)

```
SELECT last_name, first_name, job_id, (SELECT department_name
                                       FROM departments
                                       WHERE department_id =
e1.department_id) AS department_name
FROM employees e1
WHERE salary > (SELECT AVG(salary)
               FROM employees
               WHERE job_id = e1.job_id);
```

5. Theoriefragen

(2 Punkte)

Kreuzen Sie alle der folgenden Aussagen an, die wahr sind. Bitte beachten Sie, dass Sie dazu möglicherweise auch außerhalb der zur Verfügung gestellten Unterlagen recherchieren müssen.

	richtig	falsch
Virtuelle Sichten werden für die Performance-Steigerung von Abfragen eingesetzt.	☐	☒
Mit der <code>WITH CHECK OPTION</code> werden vorhandene Check-Klausel der Basistabellen aktiviert.	☐	☒
Die Ergebnismenge einer virtuellen Sicht wird bei Ausführung erstellt und nicht dauerhaft persistiert.	☒	☐
Der Einsatz von <code>MERGE</code> ist vor allem dann sinnvoll, wenn sich Relationen nicht in der zweiten Normalform befinden.	☐	☒
Bei <code>TRUNCATE</code> dürfen keine aktiven <code>FOREIGN KEY</code> Constraints auf der Tabelle vorliegen.	☒	☐
Wurden Daten bereits mit <code>COMMIT</code> festgeschrieben, können sie weder mit einem <code>SAVEPOINT</code> noch mit einem <code>ROLLBACK</code> zurückgeholt werden.	☒	☐
Mehrere DML Statements werden vor der Ausführung vom SQL Optimizer in die richtige Reihenfolge gebracht.	☐	☒
Das Flashback Feature von Oracle erlaubt Abfragen auf einen bereits vergangenen Datenstand (Historie).	☒	☐